

SET

```
In [33]: s = {}    # set & dict both define with {}  
s
```

```
Out[33]: {}
```

```
In [34]: type(s)
```

```
Out[34]: dict
```

```
In [35]: s1 = set()  
type(s1)
```

```
Out[35]: set
```

```
In [36]: s2 = {20,100,3,45}    # if you send same data type it will give in  
s2
```

```
Out[36]: {3, 20, 45, 100}
```

```
In [37]: s3 = {'z','l','c','e','f'}    # it will give ordered way  
s3
```

```
Out[37]: {'c', 'e', 'f', 'l', 'z'}
```

```
In [38]: s4 = {2,3,4,'nit',1+2j,False}  
s4
```

```
Out[38]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [39]: print(s1)  
print(s2)  
print(s3)  
print(s4)  
  
set()  
{45, 3, 100, 20}  
{'c', 'z', 'l', 'f', 'e'}  
{False, 2, 3, 4, 'nit', (1+2j)}
```

```
In [40]: s2
```

```
Out[40]: {3, 20, 45, 100}
```

```
In [41]: s2.add(30)
```

```
In [42]: s2
```

```
Out[42]: {3, 20, 30, 45, 100}
```

```
In [43]: s2.add(200)
```

```
In [44]: s2
```

```
Out[44]: {3, 20, 30, 45, 100, 200}
```

```
In [45]: s2
```

```
Out[45]: {3, 20, 30, 45, 100, 200}
```

```
In [46]: #s2[1:3] # in set indexing and slicing will not allowed
```

```
In [47]: s4
```

```
Out[47]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [48]: s5 = s4.copy()  
s5
```

```
Out[48]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [49]: s5.add(2) # duplicates are not allowed
```

```
In [50]: s5
```

```
Out[50]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [51]: s4
```

```
Out[51]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [52]: s5.clear()  
s5
```

```
Out[52]: set()
```

```
In [53]: del s5
```

```
In [54]: s4
```

```
Out[54]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [55]: s4 = {(1+2j), 2, 3, 4, False, 'nit'}  
s4
```

```
Out[55]: {(1+2j), 2, 3, 4, False, 'nit'}
```

```
In [56]: s4.remove((1+2j)) # it takes only one argument
```

```
In [57]: s4
```

```
Out[57]: {2, 3, 4, False, 'nit'}
```

```
In [58]: s3
```

```
Out[58]: {'c', 'e', 'f', 'l', 'z'}
```

```
In [59]: s3.discard('m') # in set m is not there but wont through an "error"
```

```
In [60]: s3.discard('f')
s3
```

```
Out[60]: {'c', 'e', 'l', 'z'}
```

```
In [61]: s3.pop() # it will delete random in list it delete last index
s3
```

```
Out[61]: {'e', 'l', 'z'}
```

```
In [62]: s2
```

```
Out[62]: {3, 20, 30, 45, 100, 200}
```

```
In [63]: s2.pop(3) # indexing is not allowed
s2
```

```
-----
-----
TypeError                                Traceback (most recent call
l last)
Cell In[63], line 1
----> 1 s2.pop(3) # indexing is not allowed
      2 s2

TypeError: set.pop() takes no arguments (1 given)
```

```
In [64]: s2.pop()
s2
```

```
Out[64]: {20, 30, 45, 100, 200}
```

```
In [65]: for i in enumerate(s2):
          print(i)
```

```
(0, 100)
(1, 200)
(2, 45)
(3, 20)
(4, 30)
```

```
In [66]: for i in s2:
          print(i)
```

```
100
200
45
20
30
```

```
In [67]: 100 in s2
```

```
Out[67]: True
```

```
In [68]: 1000 in s2
```

```
Out[68]: False
```

```
In [69]: s2.update(s3)
s2
```

```
Out[69]: {100, 20, 200, 30, 45, 'e', 'l', 'z'}
```

SET OPERATION

```
In [70]: s6 = {1,2,3,4,5}
s7 = {4,5,6,7,8}
s8 = {8,9,10}
```

```
In [71]: s6.union(s7) # duplication is NOT allowed
```

```
Out[71]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [72]: s6.union(s7,s8)
```

```
Out[72]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
In [73]: s6 | s7
```

```
Out[73]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [74]: s6 | s7 | s8
```

```
Out[74]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
In [75]: print(s6)
print(s7)
print(s8)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

```
In [76]: s6.intersection(s7) # common element will display
```

```
Out[76]: {4, 5}
```

```
In [77]: s6.intersection(s8)
```

```
Out[77]: set()
```

```
In [78]: s7.intersection(s8)
```

```
Out[78]: {8}
```

```
In [79]: s6 & s7
```

```
Out[79]: {4, 5}
```

```
In [80]: print(s6)
         print(s7)
         print(s8)
```

```
{1, 2, 3, 4, 5}
```

```
{4, 5, 6, 7, 8}
```

```
{8, 9, 10}
```

```
In [81]: s6.difference(s7) # common will not print
```

```
Out[81]: {1, 2, 3}
```

```
In [82]: s6 - s7
```

```
Out[82]: {1, 2, 3}
```

```
In [83]: s6 - s8
```

```
Out[83]: {1, 2, 3, 4, 5}
```

```
In [84]: s8 - s6
```

```
Out[84]: {8, 9, 10}
```

```
In [85]: s7 - s8
```

```
Out[85]: {4, 5, 6, 7}
```

```
In [86]: s8 - s7
```

```
Out[86]: {9, 10}
```

```
In [87]: print(s6)
         print(s7)
         print(s8)
```

```
{1, 2, 3, 4, 5}
```

```
{4, 5, 6, 7, 8}
```

```
{8, 9, 10}
```

```
In [88]: s6.symmetric_difference(s7) # it will remove common elements and ot
```

```
Out[88]: {1, 2, 3, 6, 7, 8}
```

```
In [89]: s6 ^ s7
```

```
Out[89]: {1, 2, 3, 6, 7, 8}
```

```
In [90]: s11 = {1,2,3,4,5,6,7,8,9}  
s12 = {3,4,5,6,7,8}  
s13 = {10,20,30,40}
```

```
In [91]: s12.issubset(s11)
```

```
Out[91]: True
```

```
In [92]: s11.issubset(s12)
```

```
Out[92]: False
```

```
In [93]: s11.issuperset(s12)
```

```
Out[93]: True
```

```
In [94]: s11 = {1,2,3,4,5,6,7,8,9}  
s12 = {3,4,5,6,7,8}  
s13 = {10,20,30,40}
```

```
In [95]: s13.isdisjoint(s12)
```

```
Out[95]: True
```

```
In [96]: s13.isdisjoint(s11)
```

```
Out[96]: True
```

```
In [97]: s12.isdisjoint(s11)
```

```
Out[97]: False
```

```
In [98]: s12 = {1,2,3,4,5}  
s13 = {10,20,30}  
s14 = {15,25,35}
```

```
In [99]: s13.issubset(s12)
```

```
Out[99]: False
```

```
In [100]: s12.issuperset(s13)
```

```
Out[100]: False
```

```
In [101]: s11
```

Out[101... {1, 2, 3, 4, 5, 6, 7, 8, 9}

```
In [102... for i in s11:  
            print(i)
```

1
2
3
4
5
6
7
8
9

```
In [103... for i in enumerate(s11):  
            print(i)
```

(0, 1)
(1, 2)
(2, 3)
(3, 4)
(4, 5)
(5, 6)
(6, 7)
(7, 8)
(8, 9)

```
In [104... sum(s11)
```

Out[104... 45

```
In [105... min(s11)
```

Out[105... 1

```
In [106... max(s11)
```

Out[106... 9

```
In [107... list(enumerate(s11))
```

Out[107... [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9)]

```
In [108... sorted(s11)
```

Out[108... [1, 2, 3, 4, 5, 6, 7, 8, 9]

Dictionary

```
In [109... d = {}
```

```
In [110...] d
```

```
Out[110...] {}
```

```
In [111...] type(d)
```

```
Out[111...] dict
```

```
In [112...] d1 = {1:'one',2:'two',3:'three'}  
d1
```

```
Out[112...] {1: 'one', 2: 'two', 3: 'three'}
```

```
In [113...] d1.keys()
```

```
Out[113...] dict_keys([1, 2, 3])
```

```
In [114...] d1.values()
```

```
Out[114...] dict_values(['one', 'two', 'three'])
```

```
In [115...] d2 = d1.copy()  
d2
```

```
Out[115...] {1: 'one', 2: 'two', 3: 'three'}
```

```
In [116...] d1.items()
```

```
Out[116...] dict_items([(1, 'one'), (2, 'two'), (3, 'three')])
```

```
In [117...] d1[1]
```

```
Out[117...] 'one'
```

```
In [118...] keys = {'ram','b','c','d'}  
value = [10,20,30]  
mydict3 = dict.fromkeys(keys,value)  
mydict3
```

```
Out[118...] {'ram': [10, 20, 30], 'c': [10, 20, 30], 'd': [10, 20, 30], 'b': [10, 20, 30]}
```

```
In [119...] value.append(50)
```

```
In [120...] mydict3
```

```
Out[120...] {'ram': [10, 20, 30, 50],  
             'c': [10, 20, 30, 50],  
             'd': [10, 20, 30, 50],  
             'b': [10, 20, 30, 50]}
```

```
In [121...] range(10)
```



```
Out[121... range(0, 10)
```

```
In [122... list(range(0,10))
```

```
Out[122... [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [123... r = range(0,10)
r
```

```
Out[123... range(0, 10)
```

```
In [124... for i in r:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

```
In [125... for i in enumerate(r):
    print(i)
```

```
(0, 0)
(1, 1)
(2, 2)
(3, 3)
(4, 4)
(5, 5)
(6, 6)
(7, 7)
(8, 8)
(9, 9)
```

Set creation

```
In [126... myset = {1,2,3,4,5} # Set of numbers
myset
```

```
Out[126... {1, 2, 3, 4, 5}
```

```
In [127... len(myset)
```

```
Out[127... 5
```

```
In [128... my_set = {1,1,2,2,3,4,5,5} # Duplicate elements are not allowed
my_set
```

```
Out[128... {1, 2, 3, 4, 5}
```

```
In [129... myset1 = {1.79,2.08,3.99,4.56,5.45} # Set of float numbers
myset1
```

```
Out[129... {1.79, 2.08, 3.99, 4.56, 5.45}
```

```
In [130... myset2 = {'abhi','rami','reddy'}
myset2
```

```
Out[130... {'abhi', 'rami', 'reddy'}
```

```
In [131... myset3 = {10,20,(1,2,3),'abhi',1.2}
myset3
```

```
Out[131... {(1, 2, 3), 1.2, 10, 20, 'abhi'}
```

```
In [132... myset3 = {10,20, "Hola", [11, 22, 32]} # set doesn't allow mutable
myset3
```

```
-----
TypeError                                Traceback (most recent call
l last)
Cell In[132], line 1
----> 1 myset3 = {10,20, "Hola", [11, 22, 32]} # set doesn't allow m
utable items like li
      2 myset3

TypeError: unhashable type: 'list'
```

```
In [133... myset4 = set() # Creation empty set
type(myset4)
```

```
Out[133... set
```

```
In [134... my_set1 = set(('one' , 'two' , 'three' , 'four'))
my_set1
```

```
Out[134... {'four', 'one', 'three', 'two'}
```

Loop through set

```
In [135... myset ={'one','two','three','four','five','six','seven','eight'}
myset
```

```
Out[135... {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [136... myset
```

```
Out[136... {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [137... for i in myset:  
            print(i)
```

```
six  
seven  
four  
one  
eight  
two  
five  
three
```

```
In [138... for i in enumerate(myset):  
            print(i)
```

```
(0, 'six')  
(1, 'seven')  
(2, 'four')  
(3, 'one')  
(4, 'eight')  
(5, 'two')  
(6, 'five')  
(7, 'three')
```

Set membership

```
In [139... myset
```

```
Out[139... {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [140... 'one' in myset
```

```
Out[140... True
```

```
In [141... 'nine' in myset
```

```
Out[141... False
```

```
In [142... if 'three' in myset:  
            print('Three is present in set')  
        else:  
            print('Three is not present in set')
```

```
Three is present in set
```

Add and Remove

```
In [143... myset
```

```
Out[143... {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [144... myset.add('nine')  
myset
```

```
Out[144...] {'eight', 'five', 'four', 'nine', 'one', 'seven', 'six', 'three',
             'two'}
```

```
In [145...] myset.remove('nine')
myset
```

```
Out[145...] {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [146...] myset.add('nine')
myset
```

```
Out[146...] {'eight', 'five', 'four', 'nine', 'one', 'seven', 'six', 'three',
             'two'}
```

```
In [147...] myset.discard('nine')
```

```
In [148...] myset
```

```
Out[148...] {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [149...] myset.update('nine', 'ten', 'eleven')
myset
```

```
Out[149...] {'e',
             'eight',
             'five',
             'four',
             'i',
             'l',
             'n',
             'one',
             'seven',
             'six',
             't',
             'three',
             'two',
             'v'}
```

```
In [150...] del myset
```

Copy Set

```
In [151...] myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'ei
myset
```

```
Out[151...] {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [152...] myset1 = myset
myset1
```

```
Out[152...] {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [153...] id(myset) , id(myset1) # The address of both myset & myset1 will be
```

```
Out[153... (4924829568, 4924829568)
```

```
In [154... my_set = myset.copy()
```

```
In [155... my_set
```

```
Out[155... {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [156... id(my_set),id(myset) # Here we can get different address
```

```
Out[156... (4924829792, 4924829568)
```

Set operations

```
In [157... A = {1,2,3,4,5}  
B = {4,5,6,7,8}  
C = {8,9,10}
```

```
In [158... A | B,C
```

```
Out[158... ({1, 2, 3, 4, 5, 6, 7, 8}, {8, 9, 10})
```

```
In [159... A.union(B)
```

```
Out[159... {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [160... A.union(B,C)
```

```
Out[160... {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
In [161... A.intersection(B)
```

```
Out[161... {4, 5}
```

```
In [162... A & B
```

```
Out[162... {4, 5}
```

```
In [163... A & C
```

```
Out[163... set()
```

```
In [164... A.difference(B)
```

```
Out[164... {1, 2, 3}
```

```
In [165... A -B
```

```
Out[165... {1, 2, 3}
```

```
In [166... A - C
```

```
Out[166... {1, 2, 3, 4, 5}
```

```
In [167... A.symmetric_difference(B)
```

```
Out[167... {1, 2, 3, 6, 7, 8}
```

```
In [168... A.issubset(B)
```

```
Out[168... False
```

```
In [169... A.issuperset(B)
```

```
Out[169... False
```

```
In [170... A.isdisjoint(B)
```

```
Out[170... False
```

Other built in functions

```
In [171... A
```

```
Out[171... {1, 2, 3, 4, 5}
```

```
In [172... min(A)
```

```
Out[172... 1
```

```
In [173... max(A)
```

```
Out[173... 5
```

```
In [174... list(enumerate(A))
```

```
Out[174... [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
```

```
In [175... D = sorted(A, reverse = True)  
D
```

```
Out[175... [5, 4, 3, 2, 1]
```

```
In [176... D = sorted(A, reverse = False)  
D
```

```
Out[176... [1, 2, 3, 4, 5]
```

Dictionary

Dictionary is a mutable data type in Python. A python dictionary is a collection of key and value pairs separated by a colon (:) & enclosed in curly braces {}. Keys must be unique in a dictionary, duplicate values are allowed.

```
In [177... mydict = dict() # empty dictionary
mydict
```

```
Out[177... {}
```

```
In [178... mydict = {1:'one' , 2:'two' , 3:'three'} # dictionary with integer keys
mydict
```

```
Out[178... {1: 'one', 2: 'two', 3: 'three'}
```

```
In [179... mydict = dict({1:'one' , 2:'two' , 3:'three'}) # Create dictionary from a list
mydict
```

```
Out[179... {1: 'one', 2: 'two', 3: 'three'}
```

```
In [180... mydict = {'A':'one' , 'B':'two' , 'C':'three'} # dictionary with character keys
mydict
```

```
Out[180... {'A': 'one', 'B': 'two', 'C': 'three'}
```

```
In [182... mydict = {1:'one' , 'A':'two' , 3:'three'} # dictionary with mixed keys
mydict
```

```
Out[182... {1: 'one', 'A': 'two', 3: 'three'}
```

```
In [183... mydict.keys()
```

```
Out[183... dict_keys([1, 'A', 3])
```

```
In [184... mydict.values()
```

```
Out[184... dict_values(['one', 'two', 'three'])
```

```
In [185... mydict.items()
```

```
Out[185... dict_items([(1, 'one'), ('A', 'two'), (3, 'three')])
```

```
In [187... mydict = {1:'one' , 2:'two' , 'A':['asif' , 'john' , 'Maria']} # dictionary with list as value
mydict
```

```
Out[187... {1: 'one', 2: 'two', 'A': ['asif', 'john', 'Maria']}
```

```
In [188... keys = {'a' , 'b' , 'c' , 'd'}
value = 10
mydict3
mydict3 = dict.fromkeys(keys , value) # Create a dictionary from a list of keys and a value
```

```
In [189... mydict3
```

```
Out[189...] {'c': 10, 'd': 10, 'b': 10, 'a': 10}
```

```
In [194...] keys = {'a' , 'b' , 'c' , 'd'}
value = [10,20,30]
mydict3 = dict.fromkeys(keys , value) # Create a dictionary from a
mydict3
```

```
Out[194...] {'c': [10, 20, 30], 'd': [10, 20, 30], 'b': [10, 20, 30], 'a': [10, 20, 30]}
```

```
In [195...] value.append(40)
```

```
In [196...] mydict3
```

```
Out[196...] {'c': [10, 20, 30, 40],
             'd': [10, 20, 30, 40],
             'b': [10, 20, 30, 40],
             'a': [10, 20, 30, 40]}
```

Accessing Items

```
In [197...] mydict = {1:'one' , 2:'two' , 3:'three' , 4:'four'}
mydict
```

```
Out[197...] {1: 'one', 2: 'two', 3: 'three', 4: 'four'}
```

```
In [198...] mydict[1]
```

```
Out[198...] 'one'
```

```
In [199...] mydict.get(1)
```

```
Out[199...] 'one'
```

```
In [200...] mydict1 = {'Name':'Asif' , 'ID': 74123 , 'DOB': 1991 , 'job' : 'Analyst'}
mydict1
```

```
Out[200...] {'Name': 'Asif', 'ID': 74123, 'DOB': 1991, 'job': 'Analyst'}
```

```
In [201...] mydict1['Name']
```

```
Out[201...] 'Asif'
```

```
In [202...] mydict1['DOB']
```

```
Out[202...] 1991
```

Change and accessing

```
In [203...] mydict1 = {'Name':'Asif' , 'ID': 12345 , 'DOB': 1991 , 'Address' : 'New York'}
mydict1
```



```
Out[203... {'Name': 'Asif', 'ID': 12345, 'DOB': 1991, 'Address': 'Hilsinki'}
```

```
In [204... mydict1['DOB'] = 2005
mydict1
```

```
Out[204... {'Name': 'Asif', 'ID': 12345, 'DOB': 2005, 'Address': 'Hilsinki'}
```

```
In [206... dict1 = {'DOB':1995}
dict1.update()
dict1
```

```
Out[206... {'DOB': 1995}
```

```
In [207... mydict1['Job'] = 'Analyst' # Adding items in the dictionary
mydict1
```

```
Out[207... {'Name': 'Asif',
            'ID': 12345,
            'DOB': 2005,
            'Address': 'Hilsinki',
            'Job': 'Analyst'}
```

Loop through dict

```
In [208... mydict1
```

```
Out[208... {'Name': 'Asif',
            'ID': 12345,
            'DOB': 2005,
            'Address': 'Hilsinki',
            'Job': 'Analyst'}
```

```
In [211... for i in mydict1:
            print(i,':',mydict1[i])
```

```
Name : Asif
ID : 12345
DOB : 2005
Address : Hilsinki
Job : Analyst
```

```
In [ ]:
```